

Project3

请在 4 月 13 日之前于学在浙大提交

一、作业概述

本次作业以 Jupyter Notebook 形式完成，围绕约束满足问题的两大核心算法展开：AC-3（弧一致性）和带启发式的回溯搜索（FC + MRV + LCV）。学生需要填写 7 个 TODO 代码块，并在提交前确保所有验证单元都打印 OK。

部分	TODO 数	内容	状态
部分 0	0	问题定义（两张图，直接运行）	已提供
部分 1	1	is_consistent — 约束检查	待完成
部分 2	2 (3+4)	revise + ac3 — 弧一致性算法	待完成
部分 3	3 (5+6+7)	forward_checking + select_mrv_variable + order_lcv_values	待完成
部分 4	0	验证运行（直接运行，对照理论题答案）	已提供

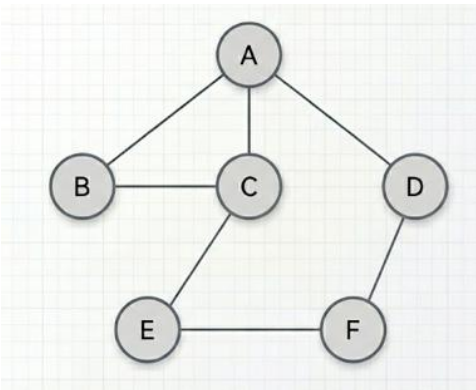
二、题目描述

问题 A：澳大利亚地图着色（Australia）

- 变量：WA, NT, SA, Q, NSW, V, T（共 7 个州/领地）
- 颜色域：{red, green, blue}
- 用途：AC-3（部分 2）

问题 B：自定义非对称图（Asymmetric）

- 变量：A, B, C, D, E, F（共 6 个）
- 颜色域：{red, green, blue}
- 用途：FC + MRV + LCV（部分 3）



问题 B 的邻接关系

```
A: [B, C, D]
B: [A, C]
C: [A, B, D, E]
D: [A, C, F]
E: [C, F]
F: [D, E]
```

三、各部分详解

部分 0：问题定义（无需修改）：

定义了两个问题字典和 `make_domains()` 辅助函数。直接运行，应打印 “Part 0 loaded OK”。**注意：domains 是可变字典，后续函数要注意是否需要深拷贝（深拷贝就是复制一个对象时，把里面嵌套的子对象也一起完整复制一份，后续互不影响），避免修改原始域。**

深拷贝：`new_domains = {v: set(d) for v, d in domains.items()}` # 深拷贝

浅拷贝：只复制最外层字典 `new_domains = domains.copy()`

TODO 1 / 7 — is_consistent:

功能：判断赋值 `var=value` 是否与所有已赋值邻居兼容。

核心逻辑：遍历 `var` 的邻居，若邻居已赋值且颜色与 `value` 相同，返回 `False`，否则返回 `True`。

验证逻辑说明（运行后可对照）：

- SA 邻居有 WA=red 和 NT=green，所以 blue 合法，red/green 不合法
- T 无邻居，任何颜色都合法

TODO 2 / 7 — revise:

功能：弧一致性的核心操作。从 `domains[Xi]` 中删除在 `domains[Xj]` 中找不到兼容值的颜色。

图着色约束：两个节点颜色不能相同，即 $x \neq y$ 时兼容。

注意事项：

- 遍历时要对 `domains[Xi]` 做拷贝 (`list(domains[Xi])`)，不能在迭代时直接修改集合
- 只要 `Xj` 中存在至少一个不等于 `x` 的颜色，`x` 就保留
- 返回值 `removed` 表示是否有变化，AC-3 依赖这个返回值决定是否重新入队

TODO 3 + 4 / 7 — ac3:

功能：完整 AC-3 算法，驱动 `revise` 对所有有向弧反复传播直到收敛。

两个 TODO 分别是：

- TODO 3：初始化队列 —— 将所有有向弧 (`Xi, Xj`) 加入 `deque`，注意每条无向边要加两个方向

- TODO 4a: 检测空域 —— revise 后若 domains[Xi] 为空, 返回 (False, None)
- TODO 4b (接 4a): 更新队列 —— 将 Xi 的其他邻居 Xk (Xk != Xj) 的弧 (Xk, Xi) 重新加入队列

验证场景说明:

- 场景 1: WA=green, V=red 强制赋值后, SA 唯一邻居覆盖了所有颜色, AC-3 应检测到不一致
- 场景 2: 仅 WA=red, NT 和 SA 域中应裁去 red, 其他颜色保留

TODO 5 / 7 — forward_checking:

功能: 赋值 var=value 后, 从未赋值邻居的域中删除 value, 若有域被删空则返回 None (提前失败)。

重要: 函数在深拷贝上操作, 不修改原始 domains。

验证要点:

- A=red 后, B/C/D 的域中应去掉 red
- 原始 domains 不应被修改 (验证单元显式检查这一点)

TODO 6 / 7 — select_mrv_variable:

功能: MRV 启发式 —— 从未赋值变量中选域最小者; 如有平局, 用 Degree 启发式打破 (未赋值邻居更多者优先)。

实现要点:

- MRV (Minimum Remaining Values): domain 越小越优先, 搜索越早失败, 剪枝越多
- Degree 打破平局: 未赋值邻居越多, 对后续变量影响越大, 优先处理能更快发现矛盾

验证要点:

- SA=red 后裁去邻居的 red, T 的域仍为 {r, g, b} (大小 3), 其余变量域大小为 2
- MRV 不应选 T, 应选任一域大小为 2 的节点

TODO 7 / 7 — order_lcv_values:

功能: LCV 启发式 —— 将候选颜色按“对邻居约束最少”排序, 淘汰数少的优先。

计算方式: 对每个候选值 v, 统计有多少个未赋值邻居的域中包含 v (若赋值 v, 这些邻居会少一个选项), 按该数升序排列。

实现要点:

- 只统计“未赋值”邻居 (已赋值的域不再参与搜索)
- sorted() 升序即满足 LCV: 淘汰最少的排前面

验证要点:

- X=red 会影响 Y (有 red) 和 Z (有 red), 共淘汰 2 个选项
- X=green/blue 只影响 Y, 共淘汰 1 个, 所以 red 应排最后

四、完整回溯求解器（已提供，无需修改）

`backtrack()` 函数将上述三个启发式函数整合，形成完整的 FC + MRV + LCV 回溯搜索器。

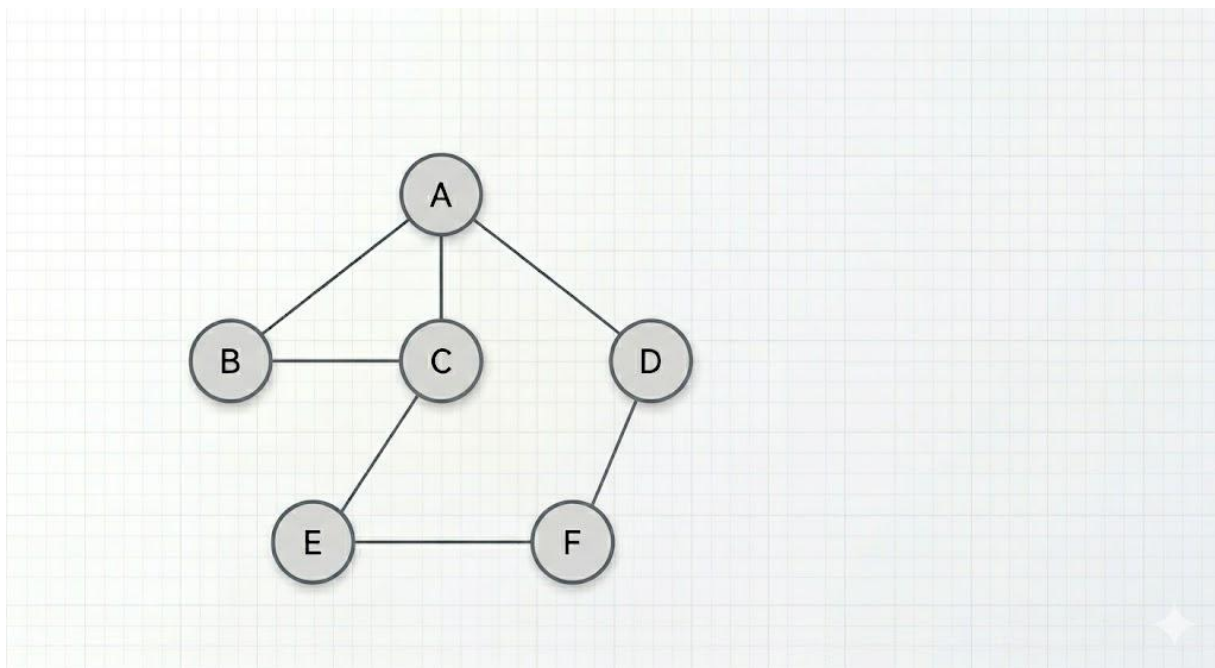
执行顺序（每次递归）：

1. `select_mrv_variable` → 选择下一个要赋值的变量
2. `order_lcv_values` → 对候选颜色排序
3. `is_consistent` → 确认候选颜色与当前赋值兼容
4. `forward_checking` → 赋值并裁剪邻居域（返回 `None` 则当前分支失败）
5. 递归调用 `backtrack` → 继续向下搜索
6. 若递归失败，撤销赋值（`del assignment[var]`），尝试下一颜色

五、部分 4：验证单元说明

验证 1：求解非对称图

调用 `solve(ASYMMETRIC)`，打印求解结果，并验证所有相邻节点颜色不同。



验证 2：FC vs AC-3 对比

在 `WA=green`, `V=red` 的前提下，分别用 FC 和 AC-3 处理，观察谁能检测到不一致。
预期结果：

- FC：只向前看一步，SA 的域被裁减为 `{blue}`（未变空），无法察觉矛盾 → 不报错
- AC-3：持续传播弧一致性，最终将某变量域裁减为空 → 报告不一致

这里的对比直接说明了 AC-3 比 FC 更强（能发现 FC 发现不了的矛盾），是理论题

Q2 的程序验证。

六、提交说明

- 完成全部 7 个 TODO (70)
- 代码必要处请提供注释 (30)
- 运行最终检查单元 (部分 4 最后的 Submission check)，确保每一项都打印 OK
- 提交 .ipynb 文件，保留所有单元格的输出
- 不得修改验证单元和 backtrack() 函数